

PROBLEMAS DE OBJETOS Y CLASES

Problema 1:

Se quiere crear la clase Tanque, que permitirá controlar los litros que hay almacenados en una cisterna. Todos los tanques que se instancien tendrán una capacidad máxima de 5000 litros.

a) Escribe el constructor y el los siguientes métodos:

- getContenido()
- agregar(...)
- sacar(...)

b) Añada un método sacaMitad() que elimina la mitad del contenido del tanque. Si el tanque está vacío no hace nada.

c) Escriba una función main que instancie y utilice un objeto t de la clase Tanque. Agregue 100 unidades al tanque. Añada código que contenga un bucle de tipo while, con sentencias que utilicen repetidamente el método sacaMitad() para reducir el contenido del tanque. La iteración deberá detenerse cuando el contenido del tanque sea menor a 1.0.

Problema 2:

a) En una carrera de velocidad participan un cierto número de atletas, cada uno de ellos tiene un nombre, número, nacionalidad y el tiempo que le tomó correr la carrera. Cree una clase Atleta que represente a un atleta. Para probar esta clase, una función main(), que haga lo siguiente: provea información acerca de dos atletas, cree dos objetos Atleta e imprima el nombre del más rápido.

b) Una carrera cubre una cierta distancia y cada carrera tiene un ganador. Cree una clase Carrera que maneje esta información. Ajuste la función main para crear un objeto de tipo Carrera y registre el nombre del atleta más rápido de la carrera.

c) Suponga que desea tener acceso a más información acerca del ganador de la carrera (por ejemplo, el tiempo que tardó o su nacionalidad). No podemos hacer esto desde el objeto carrera directamente. Necesitamos cambiar la definición de la clase tal que en lugar de almacenar el nombre del ganador, almacene un objeto Atleta. Haga este cambio muestre todos los detalles del ganador de una carrera.

d) Ahora deseamos registrar la información de todos los participantes en la carrera, podemos hacer esto añadiendo un atributo competidores a la clase Carrera, que será un array de atletas. Edite la función que determina el ganador de tal forma que ahora lo haga mirando los tiempos de cada competidor. Pruebe estos cambios.

Problema 3:

- a) Un vagón de un tren tiene 40 asientos, cada uno de ellos puede estar ocupado o vacante. El vagón puede ser de primera o segunda clase. Cree una clase Vagon para representar esta información. En el constructor se supondrá que todos los asientos inicialmente están vacantes. Escriba los métodos apropiados de acceso y actualización y un método que vaya ocupando los asientos de la siguiente forma: si el vagón es de primera hay un 10% de probabilidad que los asientos sean ocupados; si es de segunda clase hay un 40% de probabilidad que los asientos sean ocupados. Escriba una función main() que contenga un objeto Vagon, llénelo aleatoriamente e imprima el estado de cada asiento.
- b) Un tren consta de un cierto número de vagones, tiene una estación de partida y una de llegada, un cierto precio para los tickets de primera y otro para los de segunda. Cree una clase Tren que contenga esta información. Añada una función que llene los asientos de los vagones aleatoriamente. Cree un método que calcule el total de ventas de tickets. Ajuste su función main para probar lo hecho.

Problema 4:

Escriba el código de una clase Candado que modele un candado con combinación numérica (como los usados para equipaje de viaje). Al utilizar por primera vez dicho candado, puede programársele un número de 3 dígitos del 0 al 9 (deberá almacenarlos en un array) que, será la combinación de seguridad. La clase Candado deberá almacenar también información (en forma de array) del estado actual de los tres dígitos de la combinación (que en un instante dado podrá o no coincidir con los 3 dígitos programados).

Agregar un constructor que inicialice ambos arrays y añadir además los siguientes métodos:

- Un método que permita alterar alguno de los 3 dígitos (indicar posición) de la combinación actual.
- Un método, puedeAbrir, que retornará una variable booleana de valor true en caso que la cerradura pueda abrirse y, false en caso contrario.
- Un método mismaCombinacionActual que retornará una variable booleana de valor true en el caso que la combinación actual del objeto con el cual se invoca este método coincida con la combinación actual de otro Candado (deberá pasarlo como argumento al usarlo).

Deberá escribir una función main donde creará dos objetos de tipo Candado, c1 y c2. Deberá cambiar uno de los tres dígitos de la combinación actual de c1, mostrar por pantalla: un mensaje que indique si con la combinación actual de c2 que programó se puede abrir o no y otro indicando si c1 y c2 tienen (o no) programados las mismas combinaciones actuales.

Problema 5:

Haz una clase llamada Password que siga las siguientes condiciones:

- Que tenga los atributos longitud, contraseña y encriptada . Por defecto, la longitud será de 8 y no estará encriptada.
- Los constructores serán los siguientes:
 - Un constructor por defecto.
 - Un constructor con la longitud que nosotros le pasemos. Generará una contraseña aleatoria con esa longitud.
- Los métodos que implementa serán:
 - esFuerte(): devuelve un booleano si es fuerte o no, para que sea fuerte debe tener más de 2 mayúsculas, más de 1 minúscula y más de 5 números.
 - generarPassword(): genera la contraseña del objeto con la longitud que tenga.
 - encriptar(): encripta el password usando algún procedimiento que se te ocurra, y que solamente conocerán emisor y receptor de un supuesto mensaje. La forma más sencilla es el cifrado César, pero puedes usar cualquier otro que se te ocurra.
 - desencriptar(): hace lo inverso al apartado anterior.
 - Método get para contraseña y longitud.
 - Método set para longitud.

Ahora, en el main:

- Crea un array de Passwords con el tamaño que tú le indiques por teclado.
- Crea un bucle que cree un objeto para cada posición del array, con un tamaño de password aleatorio en el rango [4, 20]
- Crea otro array de booleanos donde se almacene si el password del array de Password es o no fuerte (usa el bucle anterior).
- Al final, muestra la contraseña y si es o no fuerte (usa el bucle anterior). Usa este simple formato:

contraseña1 valor_booleano1

contraseña2 valor_booleano2

...

Problema 6:

Vamos a hacer el juego de la ruleta rusa. Se trata de un número de jugadores que con un revolver con un sola bala en el tambor se dispara en la cabeza. Las clases a hacer son:

- Revolver:
 - Atributos:
 - posición actual (posición del tambor donde se dispara, puede que esté la bala o no)
 - posición bala (la posición del tambor donde se encuentra la bala, que se generarán aleatoriamente).
 - Funciones:
 - disparar(): devuelve true si la bala coincide con la posición actual
 - siguienteBala(): cambia a la siguiente posición del tambor
 - toString(): muestra información del revolver (posición actual y donde está la bala)
- Jugador:
 - Atributos
 - id (representa el número del jugador, empieza en 1)
 - nombre (Empezará con Jugador más su ID, "Jugador 1" por ejemplo)
 - vivo (indica si está vivo o no el jugador)
 - Funciones:
 - disparar(Revolver r): el jugador se apunta y se dispara, si la bala se dispara, el jugador muere.
- Juego:
 - Atributos:
 - Jugadores (conjunto de Jugadores)
 - Revolver
 - Funciones:
 - finJuego(): cuando un jugador muere, devuelve true
 - ronda(): cada jugador se apunta y se dispara, se informará del estado de la partida (El jugador se dispara, no ha muerto en esa ronda, etc.)

El número de jugadores será decidido por el usuario, pero debe ser entre 1 y 6. Si no está en este rango, por defecto será 6. En cada turno uno de los jugadores, dispara el revólver, si este tiene la bala el jugador muere y el juego termina.

Problema 7:

Crearemos una superclase llamada `Electrodomestico` con las siguientes características:

- Sus atributos son precio base, color, consumo energético (letras entre A y F) y peso.
- Por defecto, el color será blanco, el consumo energético será F, el precioBase es de 100 € y el peso de 5 kg. Usa constantes para ello.
- Los colores disponibles son blanco, negro, rojo, azul y gris. No importa si el nombre está en mayúsculas o en minúsculas.
- Los constructores que se implementaran serán
 - Un constructor por defecto.
 - Un constructor con el precio y peso. El resto, por defecto.
 - Un constructor con todos los atributos.
- Los métodos que implementara serán:
 - Métodos `get` de todos los atributos.
 - `comprobarConsumoEnergetico(char letra)`: comprueba que la letra es correcta, sino es correcta usará la letra por defecto. Se invocará al crear el objeto y no será visible.
 - `comprobarColor(String color)`: comprueba que el color es correcto, sino lo es usa el color por defecto. Se invocará al crear el objeto y no será visible.
 - `precioFinal()`: según el consumo energético, aumentará su precio, y según su tamaño, también.

A 100 €	B 80 €	C 60 €	D 50 €	E 30 €	F 10 €
---------	--------	--------	--------	--------	--------

Entre 0 y 19 kg: 10 €

Entre 20 y 49 kg: 50 €

Entre 50 y 79 kg: 80 €

Mayor que 80 kg: 100 €

Crearemos una subclase llamada `Lavadora` con las siguientes características:

- Su atributo es carga, además de los atributos heredados.
- Por defecto, la carga es de 5 kg. Usa una constante para ello.
- Los constructores que se implementaran serán:
 - Un constructor por defecto.
 - Un constructor con el precio y peso. El resto, por defecto.
 - Un constructor con la carga y el resto de atributos heredados. Recuerda que debes llamar al constructor de la clase padre.
- Los métodos que se implementara serán:
 - Método `get` de carga.

- precioFinal():, si tiene una carga mayor de 30 kg, aumentará el precio 50 €, si no es así no se incrementará el precio. Llama al método padre y añade el código necesario. Recuerda que las condiciones que hemos visto en la clase Electrodomestico también deben afectar al precio.

Crearemos una subclase llamada Television con las siguientes características:

- Sus atributos son resolución (en pulgadas) y sintonizador TDT (booleano), además de los atributos heredados. Por defecto, la resolución sera de 20 pulgadas y el sintonizador será false.
- Los constructores que se implementaran serán:
 - Un constructor por defecto.
 - Un constructor con el precio y peso. El resto por defecto.
 - Un constructor con la resolución, sintonizador TDT y el resto de atributos heredados.

Recuerda que debes llamar al constructor de la clase padre.

- Los métodos que se implementara serán:
 - Método get de resolución y sintonizador TDT.
 - precioFinal(): si tiene una resolución mayor de 40 pulgadas, se incrementara el precio un 30% y si tiene un sintonizador TDT incorporado, aumentara 50 €. Recuerda que las condiciones que hemos visto en la clase Electrodomestico también deben afectar al precio.

Ahora crea un main que:

- Crea un array de Electrodomesticos de 10 posiciones.
- Asigna a cada posición un objeto de las clases anteriores con los valores que desees.
- Ahora, recorre este array y ejecuta el método precioFinal().
- Deberás mostrar el precio de cada clase, es decir, el precio de todas las televisiones por un lado, el de las lavadoras por otro y la suma de los Electrodomesticos (puedes crear objetos Electrodomestico, pero recuerda que Television y Lavadora también son electrodomésticos). Por ejemplo, si tenemos un Electrodomestico con un precio final de 300, una lavadora de 200 y una televisión de 500, el resultado final será de 1000 (300+200+500) para electrodomésticos, 200 para lavadora y 500 para televisión.